Universidad de Buenos Aires
Facultad de Ingeniería



Sistemas Distribuidos I

Trabajo Práctico De Diseño

2do Cuatrimestre 2026

Alumnos:

- Patricio Perrone (98230)
- Juan Martin Nazarian (90206)
- José Nuñez Scarano (82159)

Índice

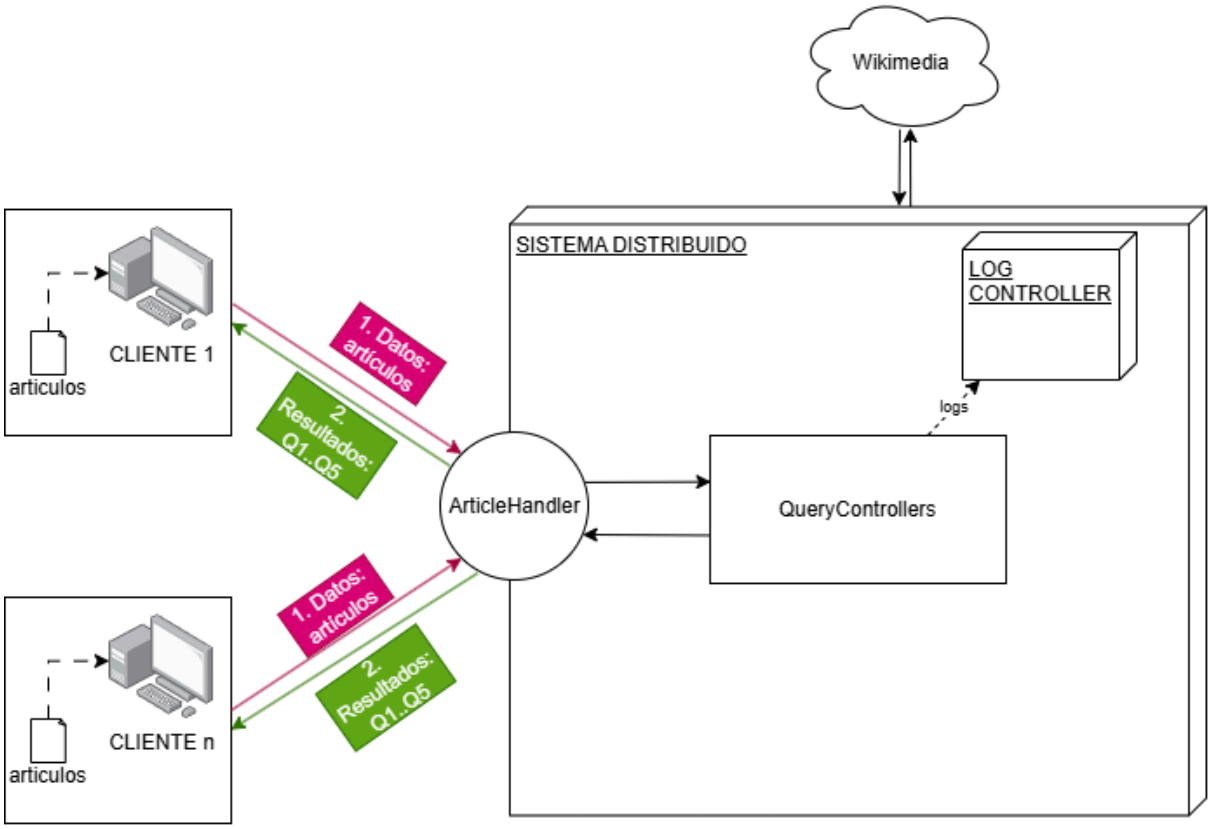
1. Alcance **3**
2. Arquitectura de Software **3**
3. Objetivos y limitaciones **4**
Objetivos 4
Restricciones 4
4. Vista Lógica **4**
5. Vista de Procesos **4**
DAG de procesamiento 5
Flujo principal 5
Reglas implementadas 6
Regla 1: Artículos escritos hasta 2020 inclusive 6
Regla 2: Artículos que incluyan ciertas palabras 8
Regla 3: Listado de palabras únicas compartidos entre artículos de ingles y frances 10
Regla 4: Top 20 12
Regla 5: Conteo superior al promedio 14
Manejo de EOF 16
6. Vista de Desarrollo **17**
Estructura de paquetes 17
common 19
7. Vista Física **20**
Diagrama de despliegue 20
8. Escenarios **21**
Escenario 1 – Envío batch de artículos. 21
Escenario 2 – Recibir resultados por partes. 22
9. Apéndices **22**
Sprint 1 — Base + Q1 (end-to-end mínimo) 22
Sprint 2 — Q2 + Q3 + Q4 (agregados y fan-out) 23
Sprint 3 — Q5 + Escalabilidad (Hito 2) 24
Sprint 4 — Tolerancia (Hito 3) 24

1. Alcance

El presente documento describe la arquitectura del sistema distribuido diseñado para analizar artículos en inglés y francés de la página Wikipedia.

2. Arquitectura de alto nivel

Visto como una abstracción de alto nivel, el sistema es capaz de atender a consultas de uno o más clientes en simultáneo, proveyendo monitoreo constante a través del registro de logs de cada componente interno, y de acceso a internet para las interacciones de clientes que lo requieran, por ejemplo: Descargar una imagen.



A nivel estructural, el flujo de datos se organiza en un esquema de procesamiento por etapas donde la información ingresa a través de una puerta de enlace central que asume el rol de distribuir el trabajo. Posteriormente, un conjunto de nodos especializados procesa los datos de manera concurrente para aplicar filtros o transformaciones, consolidando luego esos resultados parciales para unificarlos en las respuestas finales. Este diseño basado en comunicación asincrónica permite escalar la capacidad operativa del sistema, garantizando la paralelización y el aislamiento de las operaciones por cada usuario.

3.Objetivos y limitaciones

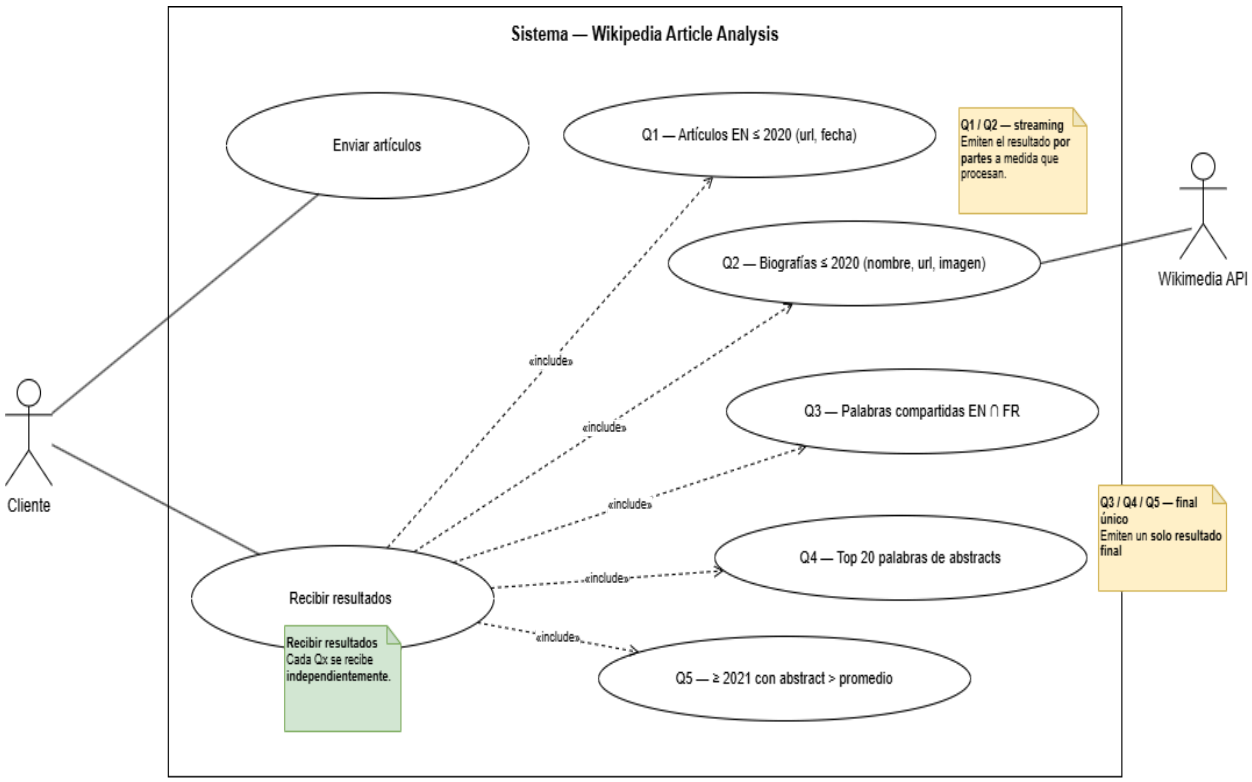
Objetivos

- Procesar grandes volúmenes de datos de manera distribuida
- Permitir escalabilidad horizontal
- Maximizar el throughput mediante paralelismo

Restricciones

- Comunicación asincrónica
- Procesamiento batch (dataset)
- Soporte de cierre controlado (graceful shutdown)

4.Escenarios



El sistema presenta interacciones con dos actores principales: el Cliente, que provee los datos de entrada, y la API de Wikimedia, utilizada como servicio externo para enriquecer la información.

Los casos de uso que resuelve el sistema son:

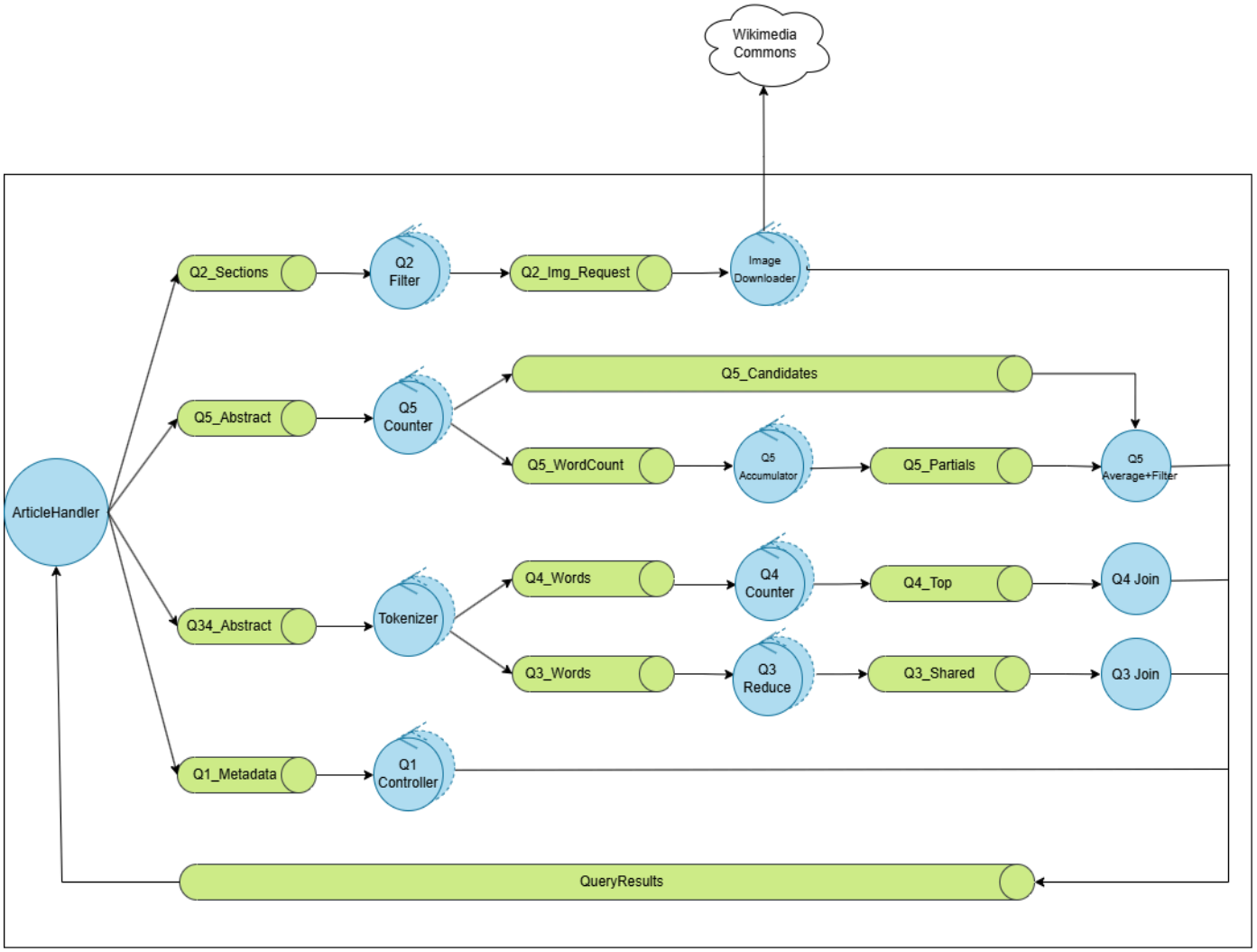
Enviar artículos: El Cliente provee y envía al sistema los conjuntos de artículos que requieren ser procesados.

Recibir resultados: El Cliente obtiene las métricas generadas por el sistema. La recepción de cada respuesta se realiza de forma independiente y se divide en dos modalidades de entrega

- **Resultados por streaming (Q1 y Q2)**: El sistema emite la información por partes a medida que procesa los datos. Esto aplica para la consulta 1, que filtra artículos en inglés publicados hasta 2020 (devolviendo URL y fecha), y para la consulta 2, que identifica biografías hasta 2020 (devolviendo nombre, URL y la imagen descargada a través de la API de Wikimedia).
- **Resultados únicos finales (Q3, Q4 y Q5)**: El sistema consolida la información y emite un único resultado al finalizar el procesamiento. Esto incluye la consulta 3 (listado de palabras compartidas entre artículos en inglés y francés), la consulta 4 (el top 20 de palabras en los resúmenes) y la consulta 5 (artículos desde 2021 cuya longitud de resumen supera el promedio histórico)

5.Vista Lógica

El sistema se compone de los siguientes componentes principales:



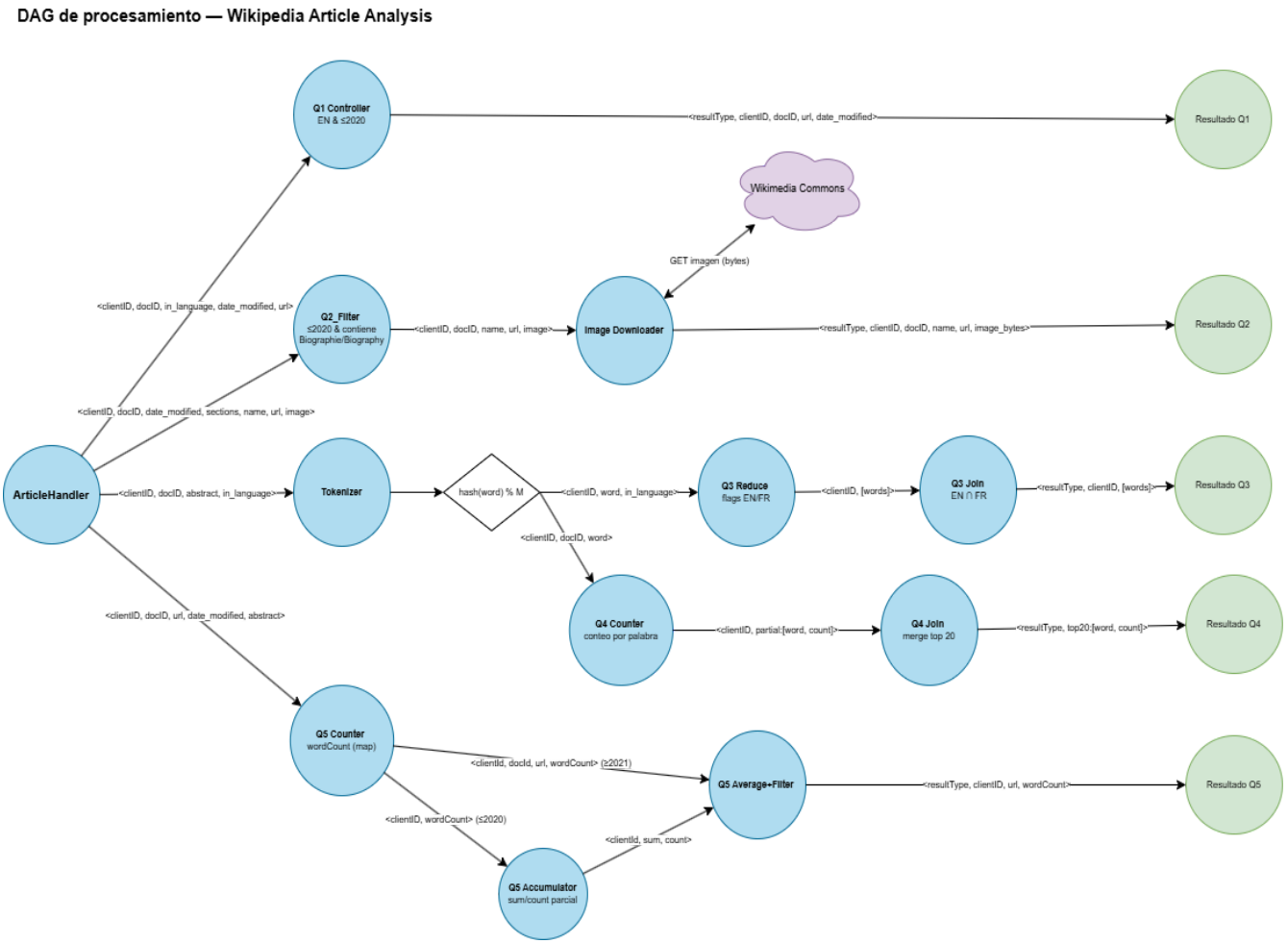
ArticleHandler: punto de contacto entre los clientes y el sistema, tanto para entrada como para salida.

QueryControllers: encargados del procesamiento distribuido de la información. Escalamiento horizontal (réplicas independientes) en todos los componentes a excepción de Article Handler, Q3 Join, Q4 Join y Q5 Average+Filter.

Middleware: los componentes se comunican internamente entre sí por medio de colas de datos

6.Vista de Procesos

A continuación, se detallan los DAGs asociados a cada requisito funcional y consulta a resolver por el sistema.

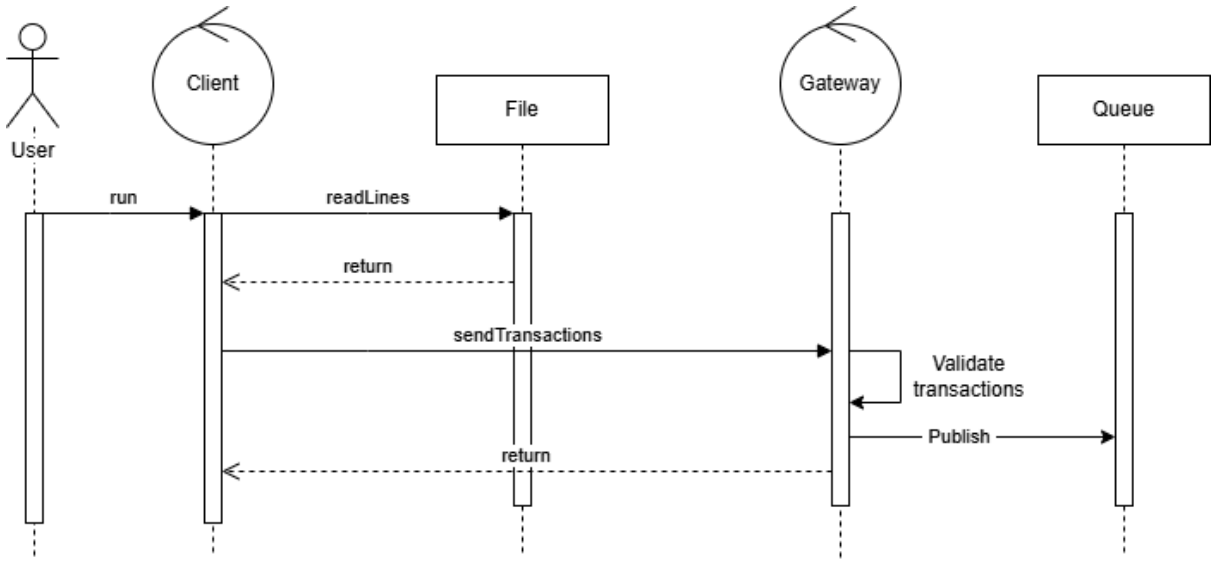


En correspondencia con el DAG previamente presentado, este diagrama detalla la ejecución concreta del sistema, mostrando cómo las distintas etapas se coordinan y cómo se materializa el paralelismo mediante workers y colas de mensajes.

Flujo principal

1. Cliente envía un bloque de artículos.
2. ArticleHandler recibe y parsea las mismas.
3. Se publican mensajes en una cola para que sean procesados los artículos.
4. Workers consumen mensajes en paralelo.
5. Cada worker aplica una regla específica.
6. Aggregators combinan resultados parciales.

7. Join genera el resultado final.



Reglas implementadas

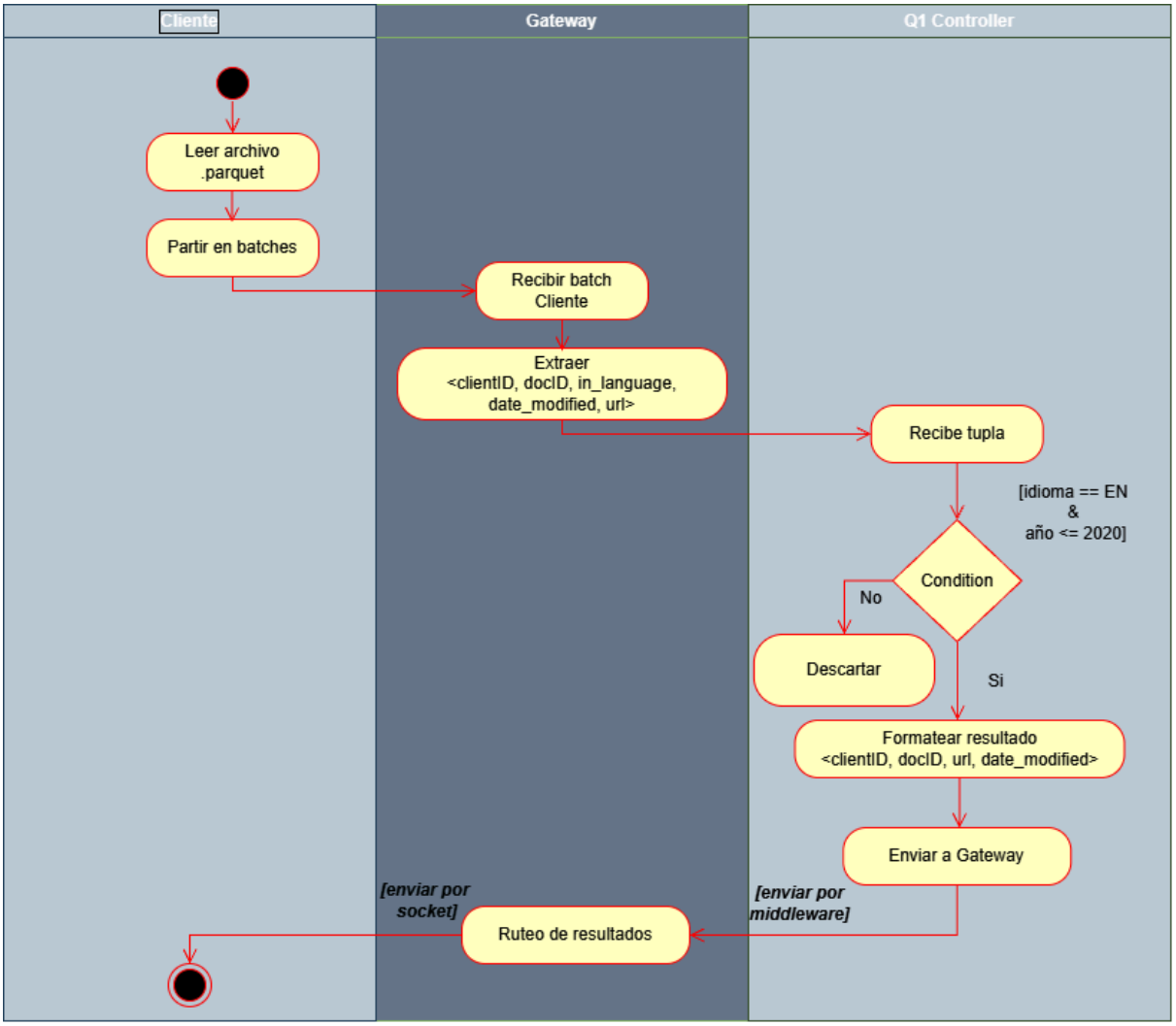
El sistema debe detectar:

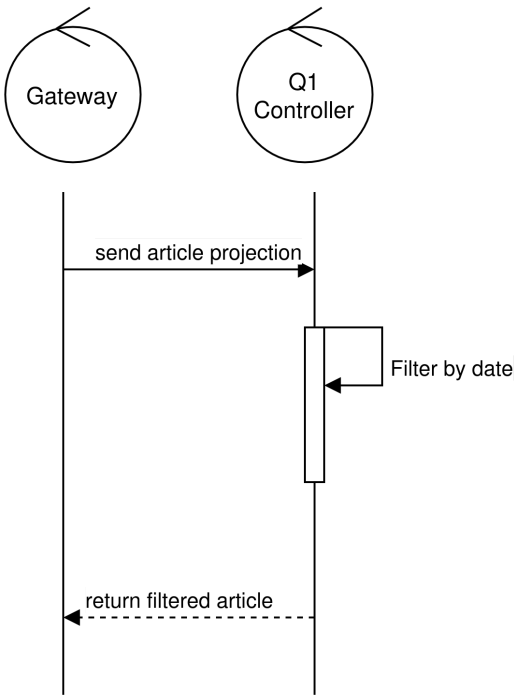
Regla 1: Artículos escritos hasta 2020 inclusive

El siguiente diagrama de secuencia detalla el flujo de procesamiento para identificar artículos escritos hasta el 2020 inclusive. El ciclo de vida de este caso de uso se compone de las siguientes interacciones:

1. **Recepción:** El **Cliente** envía el artículo al **Gateway**, el cual actúa como único punto de entrada al sistema.
2. **Encolado inicial:** El Gateway analiza y valida el paquete recibido. Una vez procesado, publica el mensaje en la cola.
3. **Filtrado:** Un **Q1 Controller** consume el artículo de la cola y evalúa la regla de negocio. Si el artículo fue escrito en inglés y fue publicado hasta el 2020 inclusive, el worker publica el resultado en la cola de salida.
4. **Retorno al cliente:** Finalmente, el Gateway consume el mensaje procesado de la cola y transmite el resultado definitivo de vuelta al cliente.

Diagrama de actividad Query 1



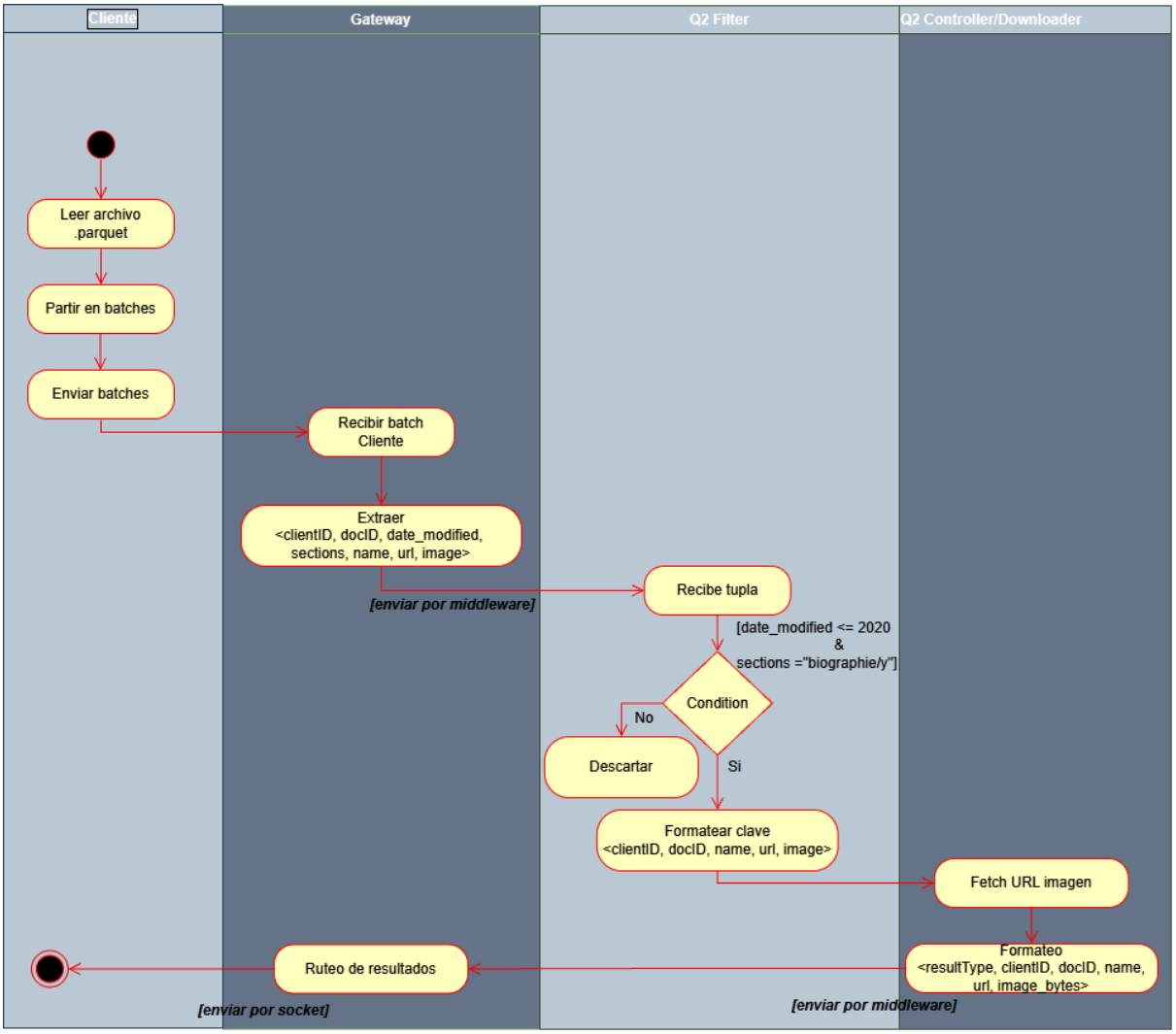


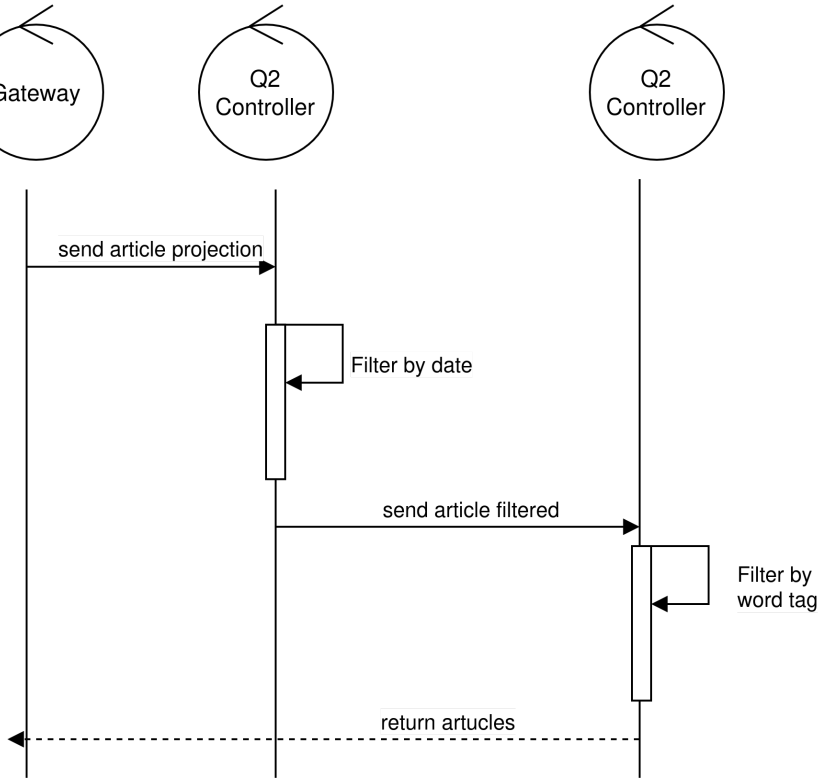
Regla 2: Artículos que incluyan ciertas palabras

El siguiente diagrama de secuencia ilustra el proceso distribuido para identificar los articulos que incluyen las palabras “biography” o “biographie” y que hayan sido publicados hasta 2020 inclusive.

1. **Recepción:** El **Cliente** envía un artículo al **Gateway**, que actúa como puerta de enlace del sistema.
2. **Encolado inicial:** El Gateway analiza el artículo y la pública en la cola genérica.
3. **Procesamiento local:** Un **Q2 Controller** consume el mensaje de la cola. Este worker filtra por fecha y encola el mensaje.
4. **Filtrado por palabras:** El nodo **Q2 Img Controller** consume el artículo publicado por el nodo de la sección anterior y filtra por palabras y envía el resultado actualizado a la cola.
5. **Retorno al cliente:** Por último, el Gateway extrae el resultado de la cola y se lo entrega al Cliente, completando el ciclo.

Diagrama de actividad Query 2



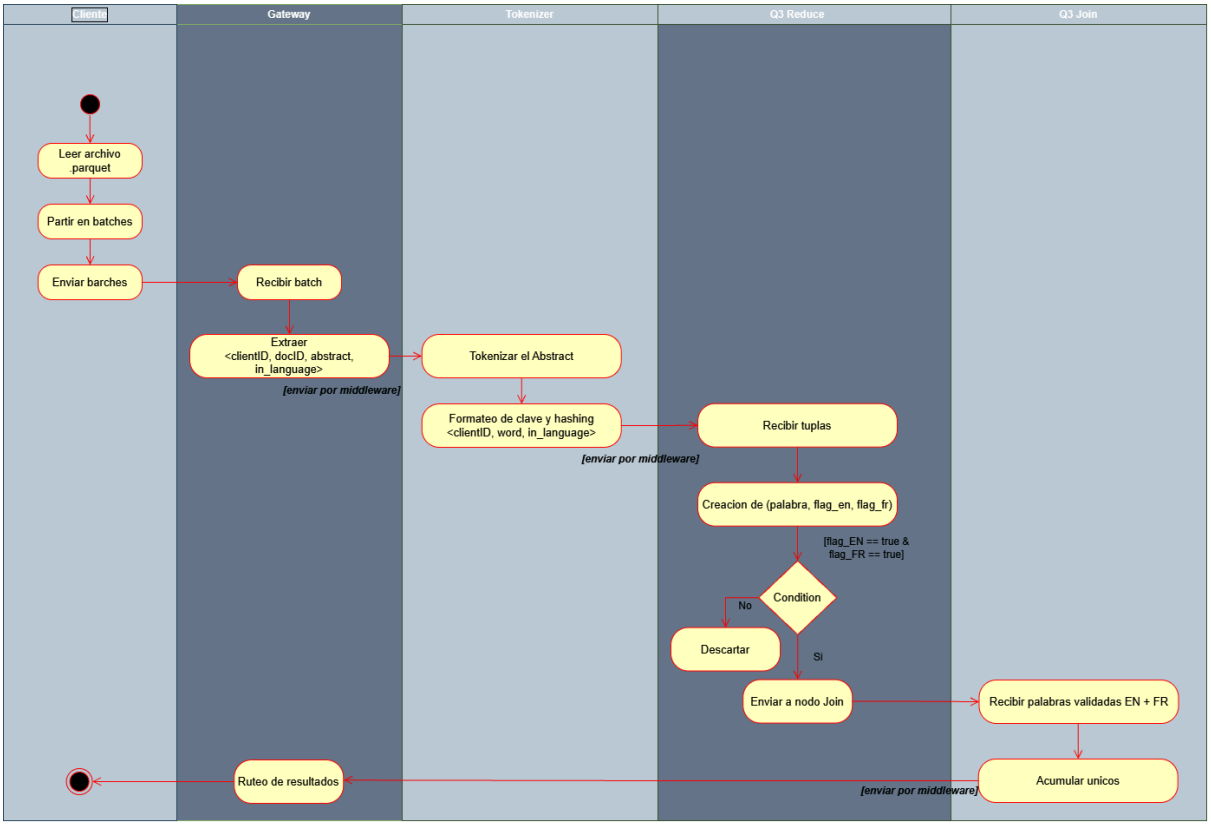


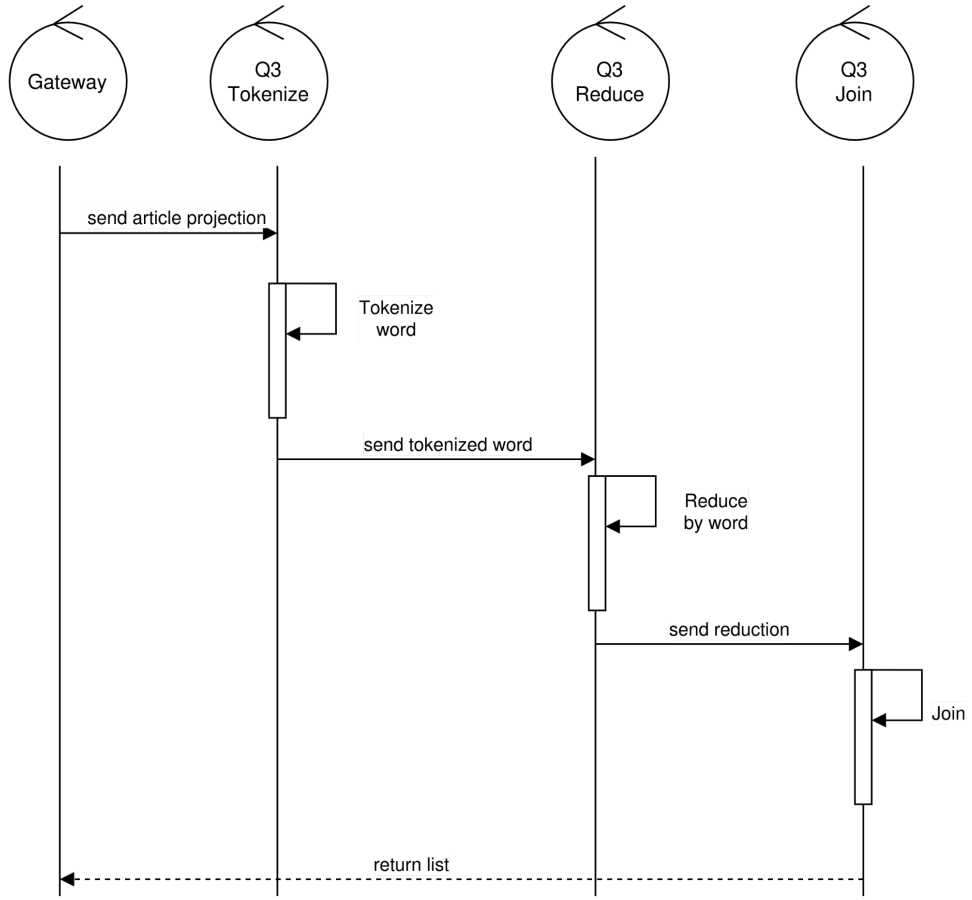
Regla 3: Listado de palabras únicas compartidos entre artículos de ingles y frances

El siguiente diagrama de secuencia detalla el flujo de la regla basada en umbrales dinámicos. Este escenario se caracteriza por la bifurcación de los datos en dos corrientes de procesamiento que luego convergen para la evaluación final:

1. **Recepción:** El **Cliente** envía un artículo al **Gateway**, que sirve como punto de entrada.
2. **Encolado inicial:** El Gateway procesa y publica el mensaje en la cola general.
3. **Tokenizacion:** Un **Q3 Tokenize** consume el artículo y tokeniza las palabras y las encola.
4. **Reduccion:** El nodo **Q3 Reduce** consume el mensaje y lo reduce por palabra y lenguaje. Cuando recibe el EOF encola los resultados.
5. **Consolidación (Join):** El nodo **Q3 Join** agrupa los resultados parciales emitidos por los distintos workers y publica los resultados globales definitivos.
6. **Retorno al cliente:** Finalmente, el Gateway consume los resultados y se los transmite al cliente.

Diagrama de actividad Query 3



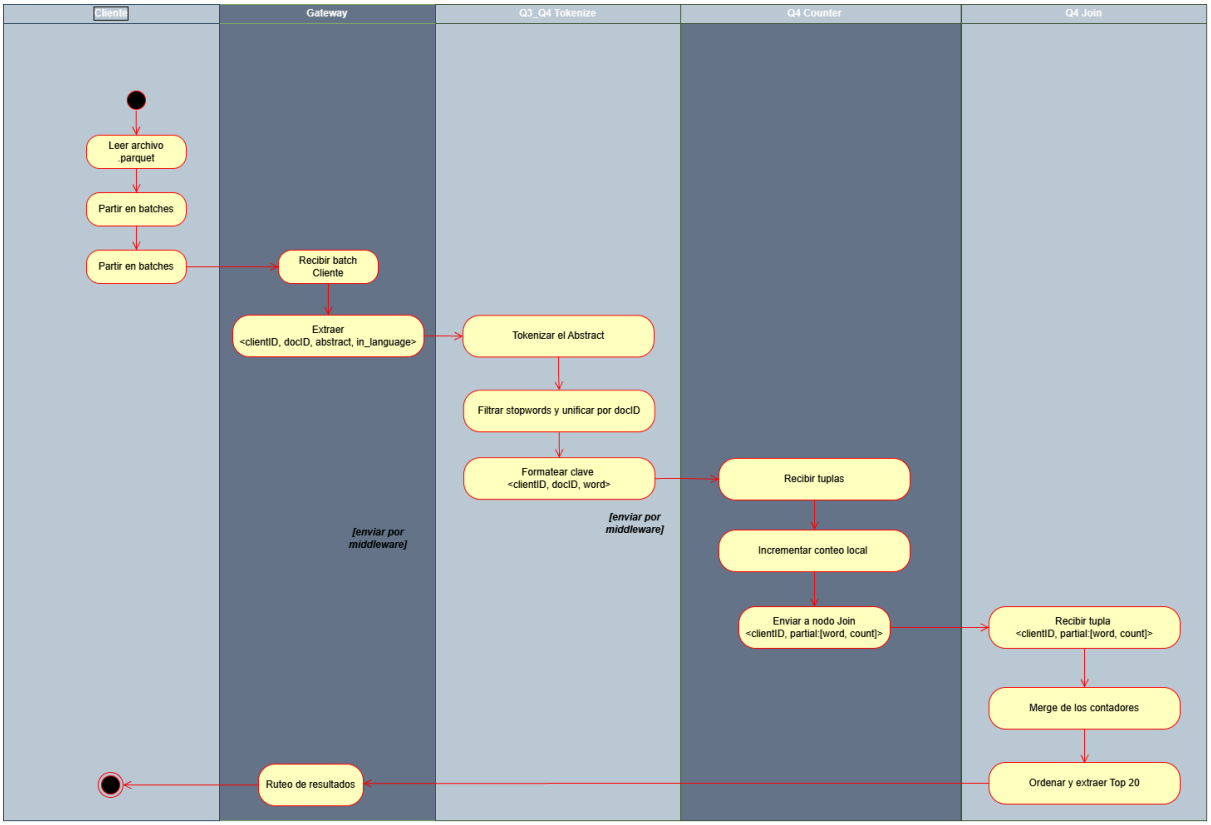


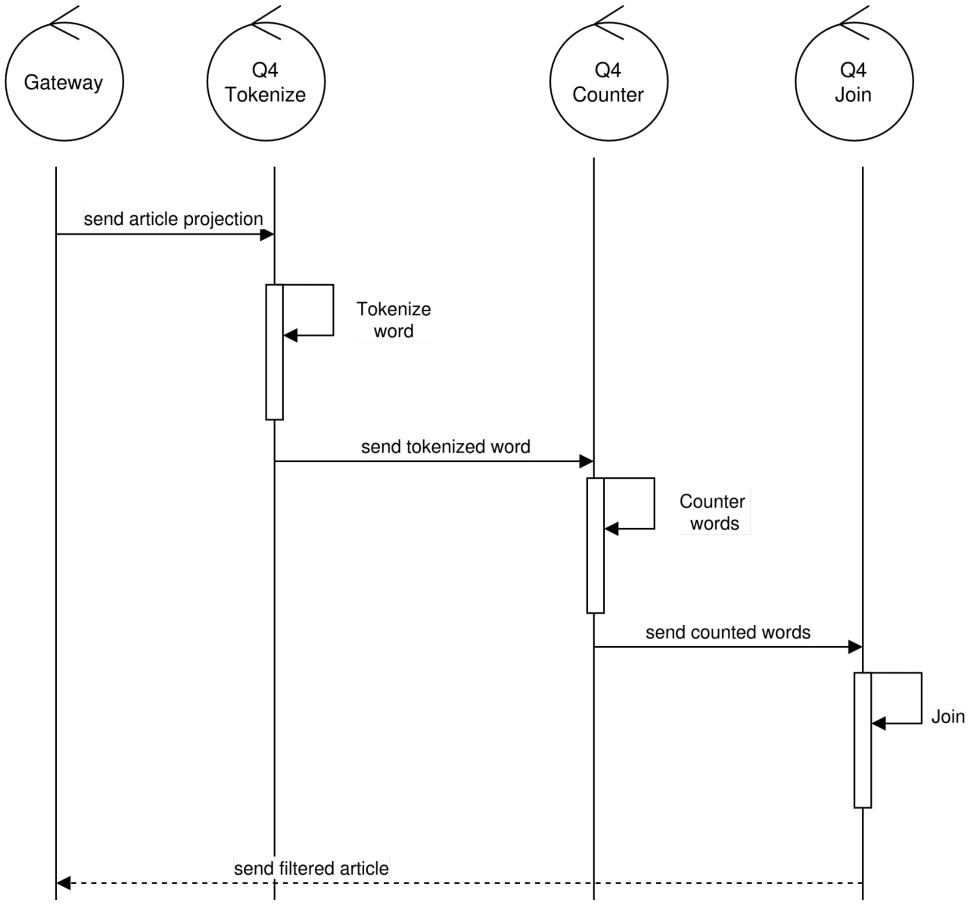
Regla 4: Top 20

El siguiente diagrama de secuencia describe el flujo de procesamiento para detectar el top 20 de pares de palabras más utilizadas entre todos los abstracts:

1. **Recepción:** El **Cliente** envía un artículo al **Gateway**, que sirve como punto de entrada.
2. **Encolado inicial:** El Gateway procesa y publica el mensaje en la cola general.
3. **Tokenizacion:** Un **Q4 Tokenize** consume el artículo y tokeniza las palabras y las encola.
4. **Contador:** El nodo **Q4 Counter** consume el mensaje y realiza una suma local por palabra. Cuando recibe el EOF encola los resultados.
5. **Consolidación (Join):** El nodo **Q4 Join** agrupa los resultados parciales emitidos por los distintos workers y publica los resultados globales definitivos.
6. **Retorno al cliente:** Finalmente, el Gateway consume los resultados y se los transmite al cliente.

Diagrama de actividad Query 4





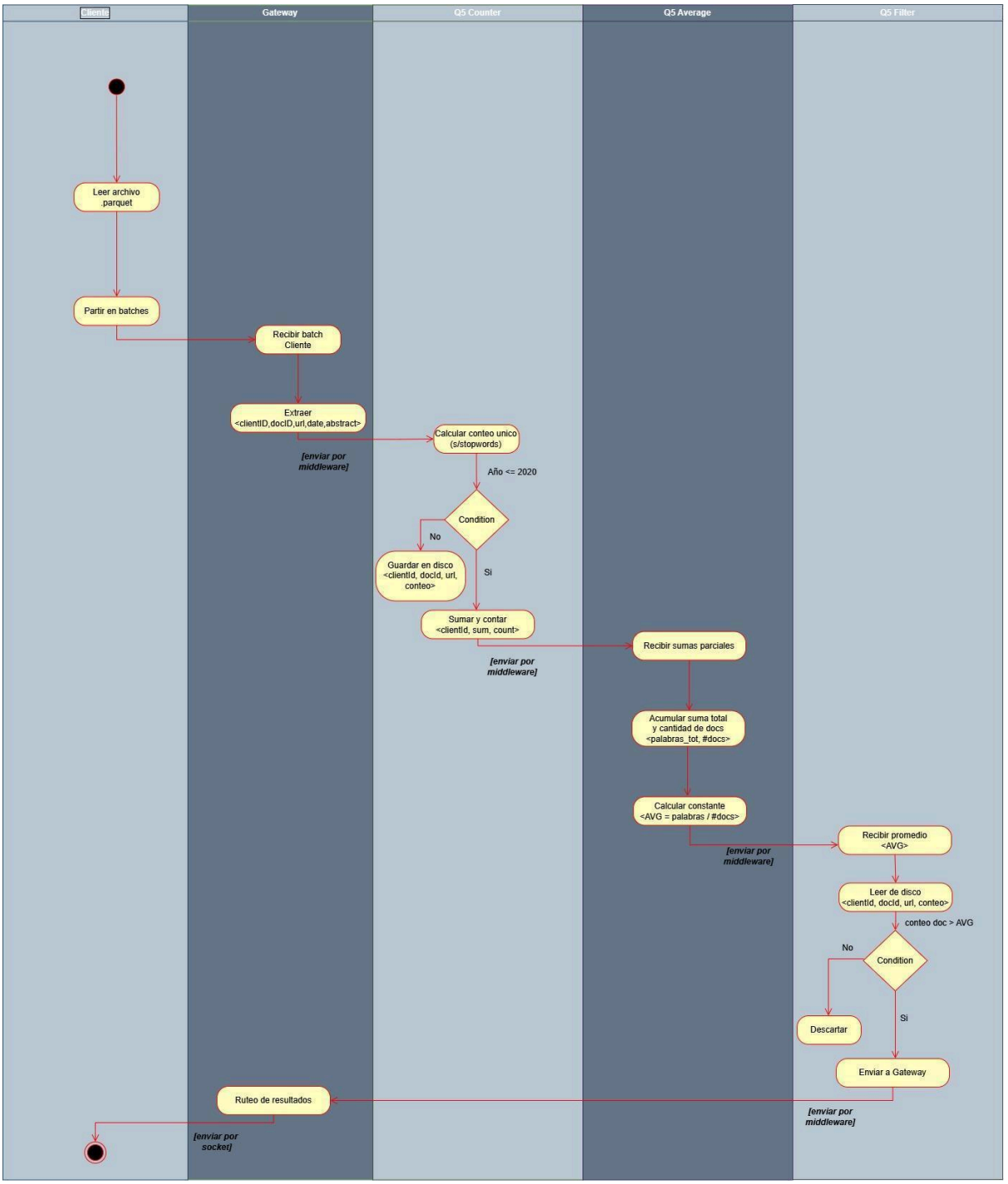
Regla 5: Conteo superior al promedio

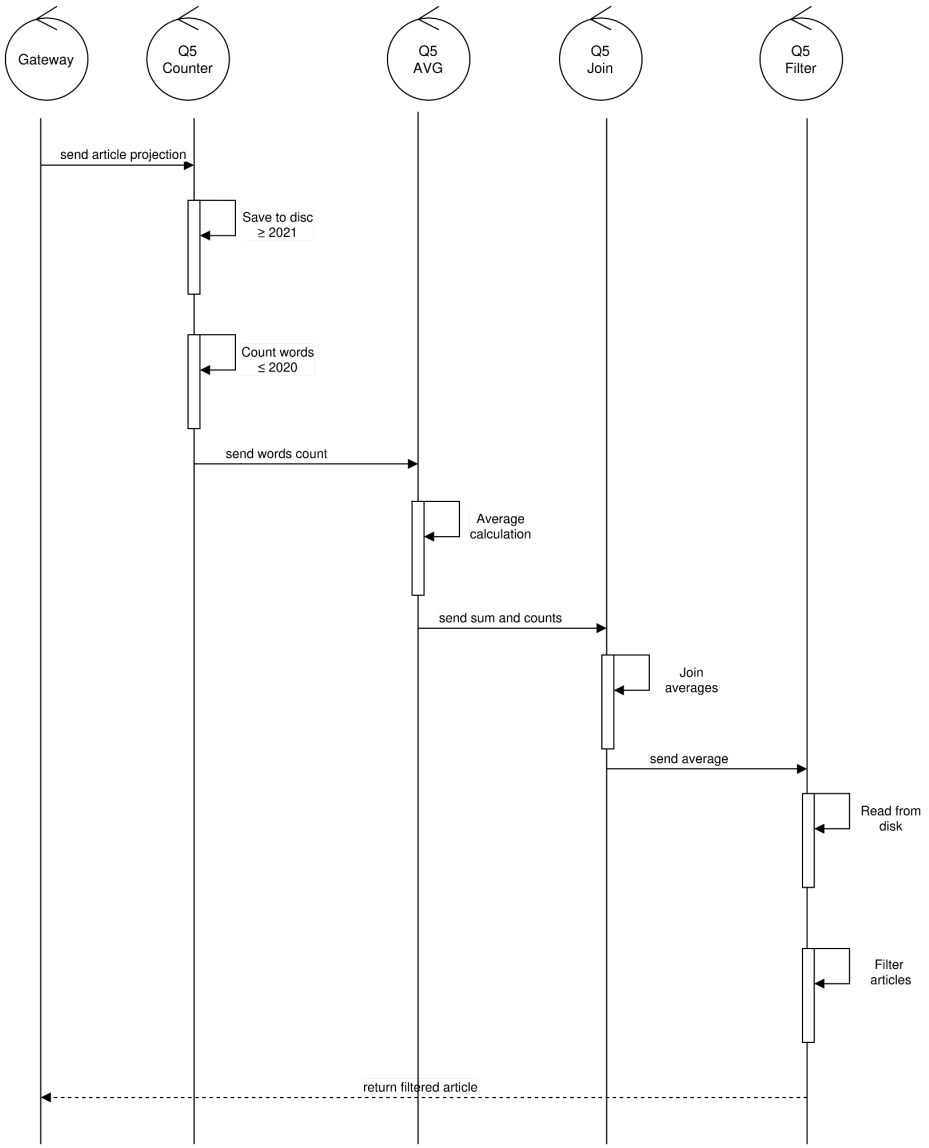
El siguiente diagrama de secuencia describe el flujo para la detección de cantidad de palabras de abstract para los artículos escritos a partir del 2021 cuya cantidad de palabras en el abstract sea superior al promedio de palabras de abstract de todos los artículos escritos hasta el 2020 inclusive

1. **Recepción:** El **Cliente** envía el artículo al **Gateway**, que actúa como el punto de entrada al sistema distribuido.
2. **Encolado inicial:** El Gateway analiza el paquete y publica el mensaje en la cola.
3. **Conteo inicial:** Un **worker** especializado consume el artículo y aplica filtros por rango de fecha y para el conteo y otro para los artículo mayores a 2021 que las guardará en disco.
4. **Promedio:** El nodo **Q5 AVG** consume el mensaje y realiza el promedio parcial. Cuando recibe el EOF publica sus resultados.
5. **Consolidación (Join):** El nodo **Q5 Join** agrupa los resultados parciales emitidos por los distintos workers y publica los resultados globales definitivos.
6. **Filtrado**: El nodo **Q5 Filter** consume los resultados consoludads y lee los resultados en disco para filtrar los artículos cuyo número de palabras en el abstract sea superior al promedio de palabras de abstract de todos los artículos escritos hasta el 2020 inclusive. Al terminar publica el artículo en la cola.

7. **Retorno al cliente:** Finalmente, el Gateway consume el resultado de la cola y transmite la información procesada de vuelta al Cliente.

Diagrama de actividad Query 5

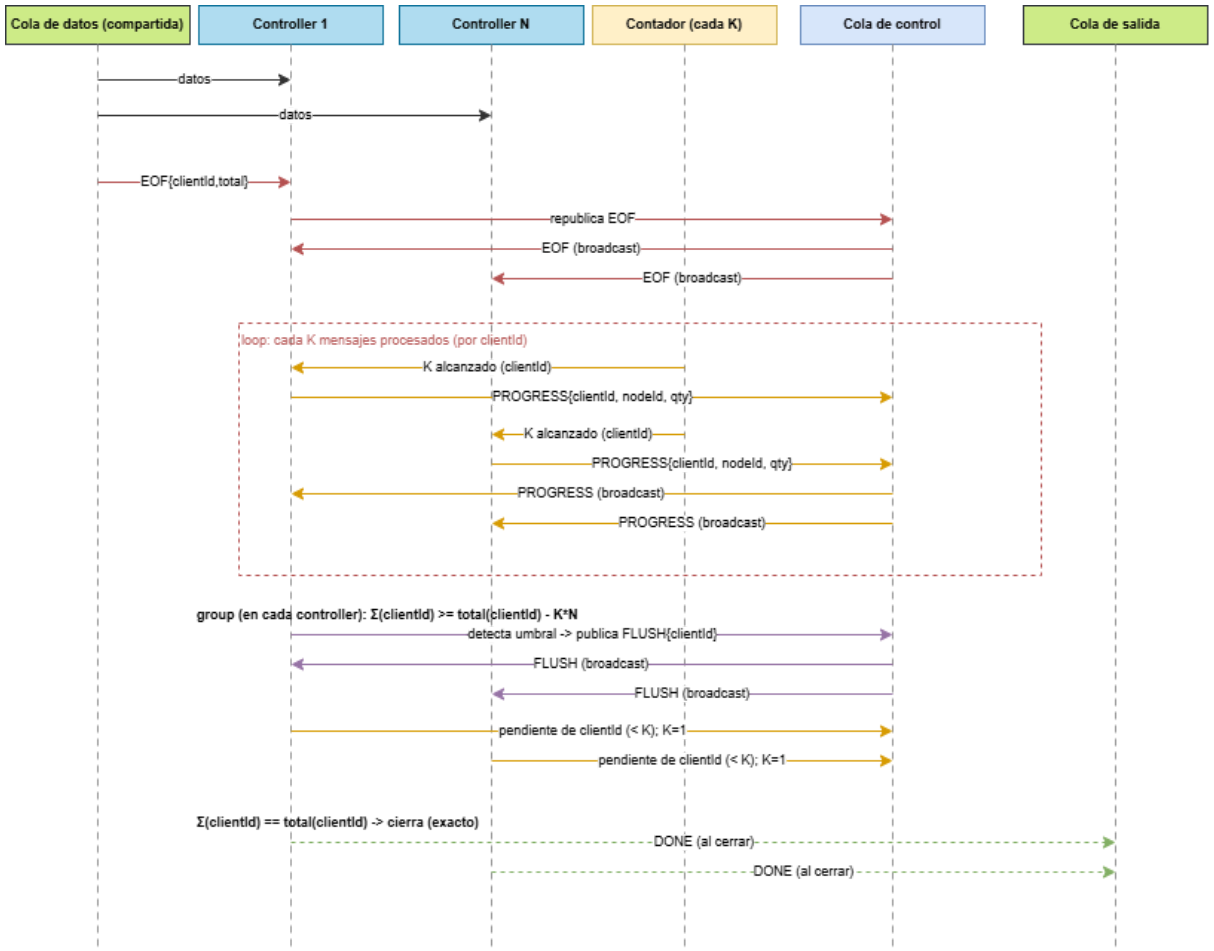



Gateway
Q5
Counter
Q5
AVG
Q5
Join
Q5
Filter
send article projection
Save to disc
≥ 2021
Count words
≤ 2020
send words count
Average
calculation
send sum and counts
Join
averages
send average
Read from
disk
Filter
articles
return filtered article

Manejo de EOF

Debido al procesamiento distribuido y asincrónico mediante colas, la recepción del último bloque de entrada no implica necesariamente que todas los artículos hayan sido procesados. Por lo tanto, el sistema requiere un mecanismo explícito de coordinación para determinar el fin real del procesamiento.

Los controllers de un tipo determinado comparten una cola de control interna para esa familia de controller. Esta cola de control puntual se encarga de la distribución de los mensajes de control necesarios para la coordinación entre las distintas réplicas.

1. 

1. ArticleHandler recibe desde un cliente el mensaje de EOF serializado con el protocolo de comunicación externo. Publica un mensaje EOF {clientId, total} en las colas de datos con las que se comunica con el resto de controllers de primer nivel. Siendo total = cuántos

mensajes se enviaron en esa cola particular de ese cliente puntual. En principio, para lo que es primer nivel, corresponde a la cantidad de artículos que envió ese cliente para el análisis.

2. Este mensaje EOF es tomado por solo una réplica, y dicha réplica republica el EOF al canal de control realizando un broadcast a todos los nodos conectados a esa cola (incluido sí mismo).
3. A medida que las réplicas procesan los datos, acumulan internamente cuánto han procesado para un cliente determinado.
4. En lugar de reportar pasado un tiempo T (ms), cada réplica publica en el canal de control un mensaje PROGRESS {clientId, controllerType, nodeID, quantity} cada K mensajes procesados de ese cliente, donde quantity es el delta procesado desde el último reporte, y K es un valor definido en la configuración de las réplicas.
5. Como el canal de control difunde cada PROGRESS a todas las réplicas de la familia, cada una puede reconstruir la misma suma global por cliente.
6. Esa reconstrucción la resuelve el middleware de grupos que corre dentro de cada controller. Mantiene, por clientId, el acumulado de los deltas recibidos y lo compara contra el total que trajo el EOF.
7. Cuando la suma reportada alcanza el umbral total − K*N (siendo N la cantidad de réplicas de ese tipo), el group de la réplica que lo detecta publica un mensaje FLUSH {clientId} en el canal de control, que se distribuye por broadcast a todas las réplicas de la familia (incluida sí misma).
8. Al recibir el FLUSH, cada réplica publica su delta pendiente (que es menor a K) y, para ese cliente, pasa a reportar cada mensaje procesado (K = 1) hasta que la etapa cierre. De esta manera, el remanente que no llegó a completar un bloque de K no queda sin reportar. Esto también cubre el caso en que no queden más mensajes en la cola: el FLUSH lo dispara la suma reportada, no la llegada de un nuevo mensaje, por lo que el pendiente se reporta igual.
9. Cuando la suma de lo procesado por todas las réplicas iguala exactamente el total esperado (Σ == total), la etapa se considera completa. El cierre es por conteo exacto y no depende de un tiempo de espera: si faltara reportar algún mensaje, la suma sería menor al total y la etapa no cerraría.
10. Al completarse la etapa, cada réplica emite su salida y un mensaje DONE {clientId, controllerType, nodeID, quantity}, donde indica cuántos mensajes de salida produjo.
11. Ese DONE lo consume la etapa siguiente: en Q3, Q4 y Q5, el Join o el Average+Filter cuentan las M réplicas para combinar los parciales y enviar el resultado. En Q1 y Q2, el ArticleHandler cuenta las N réplicas y recién entonces se informa al cliente que se finalizó con el stream.
12. En definitiva, cuando una etapa termina, cada réplica emite su DONE indicando cuántos mensajes produjo. La etapa siguiente toma esos mensajes como su propio fin de entrada y cuenta cuántas réplicas le reportaron, suma lo emitido por cada una. Con eso arma su total, ya en su propia unidad de trabajo, que pueden ser palabras, cantidad de resultados parciales según la etapa.

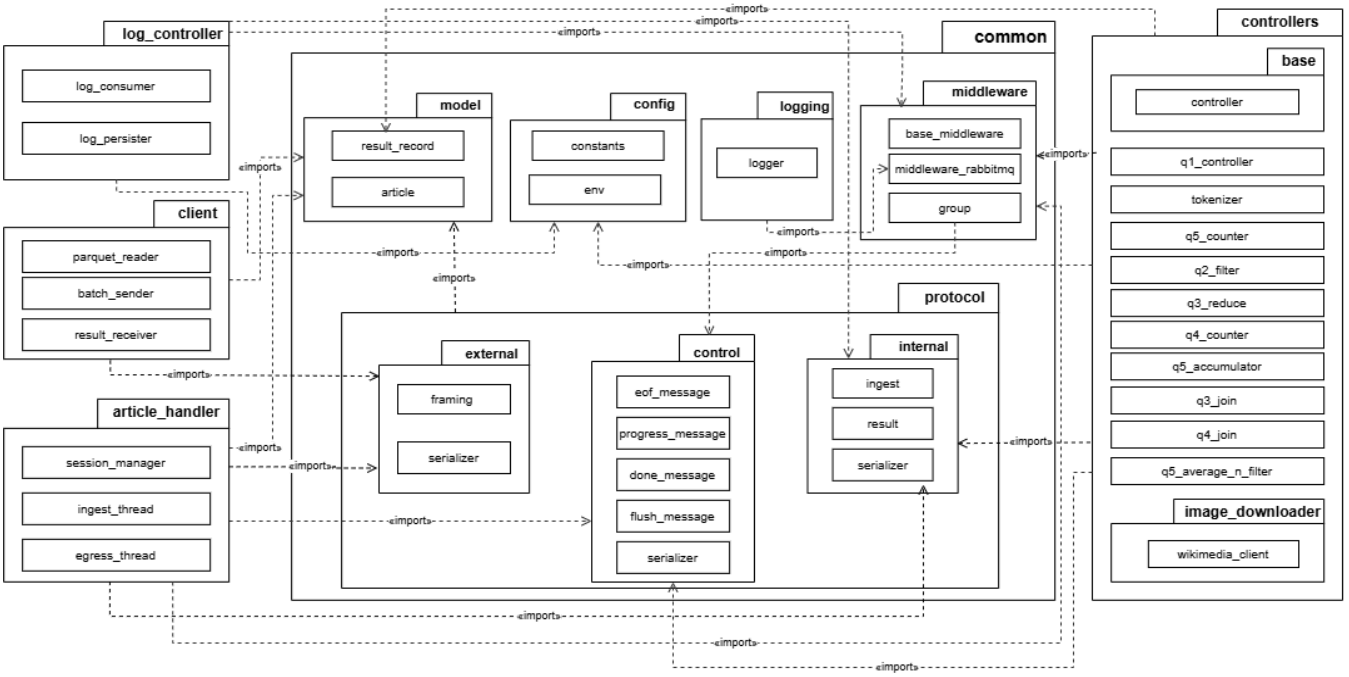
1. Vista de Desarrollo

La vista de desarrollo describe la organización del sistema a nivel de código, mostrando la descomposición en paquetes y la distribución de responsabilidades entre los distintos módulos.

Estructura de paquetes

La arquitectura del sistema fue organizada en paquetes desacoplados, donde cada componente posee una responsabilidad específica dentro del procesamiento distribuido de artículos. Esta separación permite mantener bajo acoplamiento, facilitar la escalabilidad horizontal y simplificar la incorporación de nuevas operaciones o reglas de procesamiento.

El siguiente diagrama muestra la estructura de paquetes definidos:



El código se organiza en cinco paquetes de primer nivel:

1. `client`

- `parquet_reader`: lee el `.parquet` por lotes.
- `batch_sender`: empaqueta y envía esos lotes al `article_handler`.
- `result_receiver`: recibe las respuestas por partes y las escribe.

2. `article_handler`

Frontera entre los clientes y el cluster.

- `session_manager`: valida y etiqueta la sesión (`clientId`).

- `ingest_thread`: recibe los batches y los reparte a las colas internas.
- `egress_thread`: consume los resultados y los devuelve al cliente.

3. `controllers`

Workers que procesan las queries. Cada uno recibe su stream, aplica su operación y emite su salida; ninguno se comunica directo con otro: se coordinan por el middleware.

- `base` / `controller`: clase base abstracta con el ciclo de consumo, el control (EOF/PROGRESS/DONE/FLUSH) y el cierre.
- `q1_controller`: filtra `inLanguage == "en"` y `year ≤ 2020` y emite `(url, dateModified)` en streaming (sin estado).
- `q2_filter`: filtra `year ≤ 2020` y con alguna sección de nombre exacto `biography`/`biographie`.
- `image_downloader` (con `wikimedia_client`): recibe el pedido, descarga los bytes de la imagen desde `imageUrl` (HTTP a Wikimedia) y emite `(name, url, image_bytes)` fragmentado.
- `tokenizer`: extrae del abstract las palabras no-stopword (lowercase, sin números) y las emite particionadas por `hash(palabra)`. Para Q4 deduplica por abstract. Es compartido entre Q3 y Q4.
- `q3_reduce`: por cada palabra acumula dos flags, `visto_en` y `visto_fr`. Como la palabra siempre cae en la misma partición, decide localmente; al recibir el EOF emite las palabras con `en && fr`.
- `q3_join`: reúne las M parciales (barrera M) y arma el listado final de palabras compartidas.
- `q4_counter`: cuenta en cuántos abstracts distintos aparece cada palabra (una vez por abstract, por el dedupe del tokenizer), particionado por `hash(palabra)`. Al recibir EOF emite su top 20 parcial.
- `q4_join`: recibe los M top parciales (barrera M), los fusiona y extrae el top 20 global.
- `q5_counter`: calcula `wordCount = len(extract(abstract))` (con repetición, sin stopwords) y rutea: ≤2020 a `q5_accumulator`, ≥2021 a `q5_average_n_filter` (candidatos).
- `q5_accumulator`: suma los `wordCount` y cuenta los abstracts de ≤2020 (parcial). Al procesar el EOF envía `(sum, count)`.
- `q5_average_n_filter`: único; recibe los P parciales `(sum, count)` (barrera P) y los candidatos ≥2021, calcula el promedio global, filtra `wordCount > avg` y emite `(url, wordCount)` como resultado final (`isLastPart = true`).

4. `common`

`model`

- `article`: el documento normalizado del dataset (`docId, name, url, year, inLanguage, sections, abstract, imageUrl`).
- `result_record`: una parte de resultado (`resultType, clientId, docId, partId, isLastPart, isSessionEnd, payload`).

`middleware`

Transporte y coordinación.

- `base_middleware`: interfaz/base del transporte.
- `middleware_rabbitmq`: implementación sobre RabbitMQ (`pika`): abrir conexión/canal, declarar y publicar/consumir colas y exchanges, ACK/NACK.
- `group`: primitivas de coordinación de grupos, construidas sobre el broker:
    - `reduce`: combinar parciales por clave entre réplicas.
    - `gather/broadcast`: reunir o difundir un valor global.
    - `barrier`: esperar que N/M réplicas cierren antes de continuar. Oculta la complejidad de la comunicación por grupos a los controllers.
    - `flush`: detectar el umbral (`Σ ≥ total − K*N`) y difundir `FLUSH{clientId}` para que cada réplica reporte su pendiente.

`logging`

- `logger`: publica los logs en la logs queue.

`config`

- `constants`: constantes del sistema.
- `env`: lectura de variables de entorno.

`protocol`

- `external` (entre cliente y `article_handler`):
    - `framing`: safe socket (`send_all`/`recv_all`, manejo de short write/read) con prefijo de largo para delimitar mensajes sobre TCP.
    - `serializer`: serialización de paquetes externos.
- `internal` (entre procesos):
    - `ingest`: define los mensajes de ingesta (`article_handler` a workers), con la proyección según tipo (para las distintas queries).
    - `result`: define los mensajes de retorno (workers al `article_handler`), las partes de resultado.
    - `serializer`: codificación de los DTOs internos.
- `control` (coordinación):
    - `eof_message`, `progress_message`, `done_message`, `flush_message`: mensajes de control (EOF, PROGRESS, DONE, FLUSH).
    - `serializer`: codificación de los mensajes de control.

5. `log_controller`

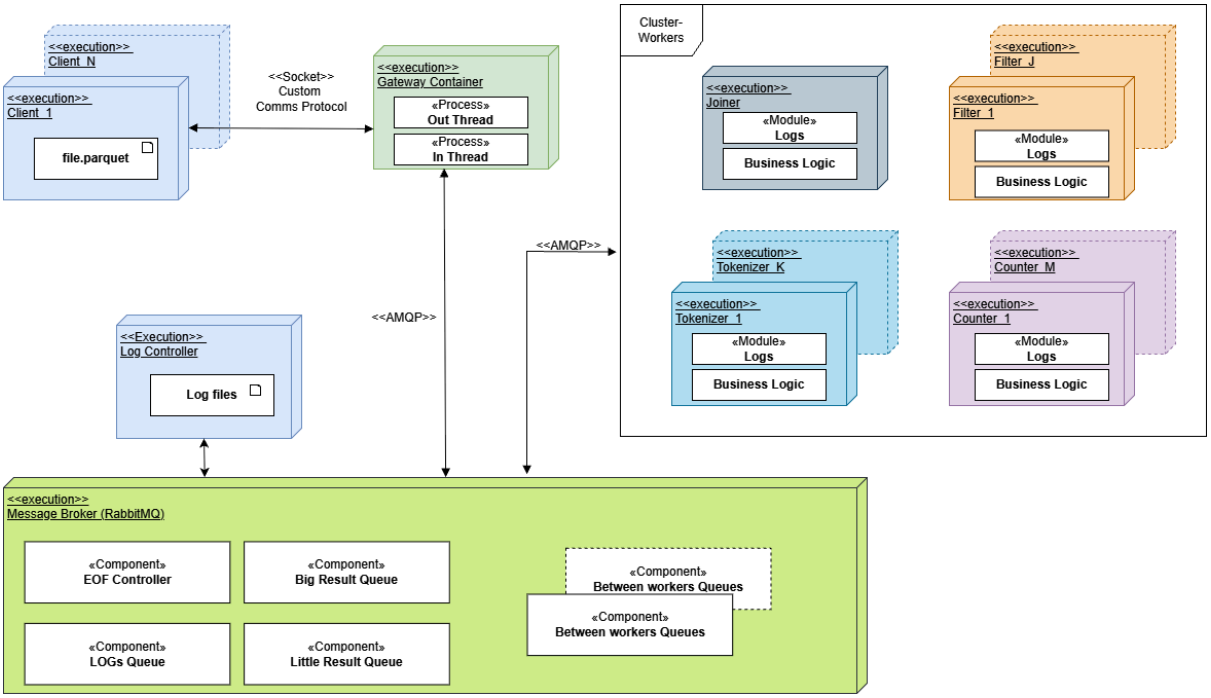
Servicio que consume y persiste los logs.

- `log_consumer`: consume la logs queue.
- `log_persister`: escribe los logs en archivos.

2. Vista Física

La vista física describe la distribución de los componentes del sistema en la infraestructura de ejecución, así como las conexiones entre ellos.

Diagrama de despliegue



El diagrama de despliegue describe la distribución física de los componentes del sistema sobre la infraestructura de ejecución, modelando cada unidad de cómputo como un nodo UML con sus artefactos desplegables y las conexiones entre ellos.

Nodo Client {1..*} representa los hosts de los usuarios que alimentan el sistema. Puede existir más de una instancia simultánea, cada una ejecutando el mismo contenedor docker.

Cada cliente lee uno o más archivos ***.parquet*** del sistema de archivos local y los transmite al Gateway mediante una conexión TCP/IP a través de un Socket

Nodo Gateway {1} actúa como único punto de entrada al sistema distribuido.

Sistema dentro del cual corre el proceso Gateway, responsable de recibir los artículos enviadas por los clientes, parsearlas, validarlas y publicarlas en el broker de mensajería correspondiente a cada query.

Una vez que el sistema finaliza el procesamiento, el Gateway recibe los resultados finales desde el Nodo Join y los devuelve al cliente correspondiente, cerrando el ciclo TCP/IP.

Nodo Middleware {1} aloja el broker RabbitMQ.

Este nodo centraliza toda la comunicación asincrónica del sistema a través de AMQP.

Contiene el conjunto completo de queues y exchanges que coordinan el flujo de datos entre los demás nodos. Ningún nodo se comunica directamente con otro: todo el tráfico pasa por este broker.

Nodo Workers {1..*} Son los nodos de procesamiento principal y puede escalar horizontalmente con múltiples instancias.

Hay varios tipos de nodos que tienen distintas responsabilidades e implementaciones de código:

- Tokenizer
- Joiner
- Filter
- Joiner

Nodo Log controller {1} Es el encargado de recibir de los diferentes nodos sus mensajes de log, útiles para el monitoreo del sistema. También es el que garantiza el guardado de los mismos particionados.

3. Apéndices

División de trabajo en los sucesivos sprints:

Sprint 1 — Base + Q1 (end-to-end mínimo)

#	Tarea	Paquete	Owner
1.1	Framing (safe socket + length-prefixed)	`common/protocol/external`	Patricio
1.2	Serializer externo (uint/string/bool/bytes/list)	`common/protocol/external`	Patricio
1.3	Protocolo A/ingesta + serializer interno	`common/protocol/internal`	Juan
1.4	Protocolo C/result + serializer interno	`common/protocol/internal`	Juan
1.5	Broker RabbitMQ (colas/exchanges, ack/nack)	`common/middleware`	Juan
1.6	Model (Article, ResultRecord)	`common/model`	Jose
1.7	Cliente (parquet_reader, batch_sender, result_receiver)	`client`	Patricio

1.8	Gateway (session_manager, ingest_thread, egress_thread)	`gateway`	Patricio
1.9	`q1_controller` (EN ≤2020 → url, fecha)	`controllers`	Jose
1.10	Colas nombradas + `docker-compose` base (+rabbitmq)	infra	Jose
1.11	Test E2E Q1 (cliente manda, Q1 responde)	tests	Todos

DoD: levantar con `docker-compose`, mandar un `.parquet` y recibir Q1 correctamente.

Sprint 2 — Q2 + Q3 + Q4 (agregados y fan-out)

#	Tarea	Paquete	Owner
2.1	`q2_filter` (≤2020 + sección biografía)	`controllers`	Patricio
2.2	`q2_img` (descarga bytes + chunking)	`controllers`	Patricio
2.3	`tokenizer` compartido (emite palabras + `wordCount`)	`controllers`	Juan
2.4	`group` (reduce / gather / broadcast / barrier)	`common/middleware`	Juan
2.5	Control/EOF: exchange fanout + conteo N/M	`common/middleware`	Juan
2.6	`q3_reduce` (flags EN/FR) + `q3_join` (intersección)	`controllers`	Patricio
2.7	`q4_counter` (conteo) + `q4_join` (top 20)	`controllers`	Patricio
2.8	Particionado `hash(palabra) % M`	`controllers`	Jose
2.9	Test E2E Q2 / Q3 / Q4	tests	Todos

DoD: Q2 (imágenes), Q3 (intersección) y Q4 (top 20) correctos, con barreras por EOF.

Sprint 3 — Q5 + Escalabilidad (Hito 2)

#	Tarea	Paquete	Owner
3.1	`q5_counter` (`wordCount` + ruteo ≤2020 / ≥2021)	`controllers`	Jose
3.2	`q5_avg` (parcial) + `q5_join` (promedio global + broadcast)	`controllers`	Jose
3.3	Spill a disco (≥2021) + `q5_filter`	`controllers`	Jose
3.4	Escalado: réplicas por variable en `docker-compose`	infra	Patricio
3.5	Gateway escalable (sharding por `clientId`)	`gateway`	Patricio
3.6	Streaming cliente→sistema y sistema→cliente	`client`/`gateway`	Juan
3.7	Validación de escalabilidad (multi-réplica)	tests	Todos

DoD: Q5 correcto; sistema con ≥50% de tipos de controller escalables y streaming real.

Sprint 4 — Tolerancia (Hito 3)

#	Tarea	Owner
4.1	Exactly-once hacia el cliente (idempotencia con `docId`/`partId`)	Juan
4.2	Graceful shutdown completo ante SIGTERM (todos los procesos)	Patricio
4.3	Limpieza de recursos y archivos de spill al resolver/fallar	Jose
4.4	Escenario Chaos Monkey (matar containers + re-instanciar)	Jose
4.5	Automatizar escenarios de prueba (incl. Chaos Monkey)	Patricio
4.6	README final + `Makefile`	Juan